

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE BERRECHID

Filière : Ingénierie des systèmes d'information et Big Data

Année universitaire : 2025–2026

---

## RAPPORT DE PROJET

---

# Détection automatique d'anomalies financières avec BI augmentée

Conception d'une plateforme intelligente pour l'analyse,  
la détection et l'investigation des transactions suspectes

**Réalisé par :** Bensaid Malak et Amar Chai-  
maa

**Encadré par :** Pr. Hrimech Hamid

**Module :** Business Intelligence et Entre-  
pôt de données

---

**Juin 2026**

# Résumé

Ce projet propose une plateforme de détection automatique d'anomalies dans des transactions financières. L'objectif n'est pas seulement de produire une classification binaire, mais de fournir un outil complet d'aide à la décision : ingestion de données, création de variables pertinentes, détection par plusieurs modèles, exposition des prédictions par une API REST, visualisation dans un tableau de bord décisionnel et collecte du retour des analystes.

La solution combine deux approches non supervisées : *Isolation Forest* et un autoencodeur basé sur un réseau de neurones. Un modèle d'ensemble agrège leurs scores afin de renforcer la robustesse de la détection. Une interface Streamlit complète ce dispositif : elle permet de consulter des indicateurs, d'investiguer les anomalies, de soumettre une transaction manuellement et d'enregistrer une annotation humaine. Cette boucle de retour constitue la dimension de *Business Intelligence augmentée* du projet.

# Table des matières

|                                           |          |
|-------------------------------------------|----------|
| <b>Résumé</b>                             | <b>i</b> |
| <b>1 Introduction</b>                     | <b>1</b> |
| 1.1 Contexte . . . . .                    | 1        |
| 1.2 Problématique . . . . .               | 1        |
| 1.3 Objectifs . . . . .                   | 1        |
| 1.4 Périmètre . . . . .                   | 1        |
| <b>2 Analyse fonctionnelle</b>            | <b>2</b> |
| 2.1 Acteurs . . . . .                     | 2        |
| 2.2 Fonctionnalités principales . . . . . | 2        |
| <b>3 Architecture de la solution</b>      | <b>3</b> |
| 3.1 Vue générale . . . . .                | 3        |
| 3.2 Organisation du dépôt . . . . .       | 3        |
| 3.3 Technologies utilisées . . . . .      | 4        |
| <b>4 Données et préparation</b>           | <b>4</b> |
| 4.1 Structure d'une transaction . . . . . | 4        |
| 4.2 Validation . . . . .                  | 5        |
| 4.3 Feature engineering . . . . .         | 5        |
| <b>5 Modèles de détection</b>             | <b>6</b> |
| 5.1 Isolation Forest . . . . .            | 6        |
| 5.2 Autoencodeur . . . . .                | 7        |
| 5.3 Modèle d'ensemble . . . . .           | 7        |
| 5.4 Explicabilité . . . . .               | 7        |
| <b>6 API REST</b>                         | <b>8</b> |
| 6.1 Rôle de l'API . . . . .               | 8        |
| 6.2 Endpoints . . . . .                   | 8        |
| <b>7 Dashboard BI augmenté</b>            | <b>8</b> |

|           |                                            |           |
|-----------|--------------------------------------------|-----------|
| 7.1       | Objectif . . . . .                         | 8         |
| 7.2       | Pages disponibles . . . . .                | 9         |
| 7.3       | Boucle d'active learning . . . . .         | 9         |
| <b>8</b>  | <b>Illustrations de l'exécution</b>        | <b>10</b> |
| <b>9</b>  | <b>Ingestion temps réel et persistance</b> | <b>15</b> |
| 9.1       | Simulation temps réel . . . . .            | 15        |
| 9.2       | Intégration Snowflake . . . . .            | 16        |
| 9.3       | Déploiement avec Docker . . . . .          | 16        |
| <b>10</b> | <b>Installation et exécution</b>           | <b>16</b> |
| 10.1      | Prérequis . . . . .                        | 16        |
| 10.2      | Entraînement . . . . .                     | 17        |
| 10.3      | Démarrage manuel . . . . .                 | 17        |
| 10.4      | Démarrage avec Docker . . . . .            | 17        |
| <b>11</b> | <b>Validation et tests</b>                 | <b>17</b> |
| 11.1      | Tests automatiques . . . . .               | 17        |
| 11.2      | Vérification du service . . . . .          | 18        |
| <b>12</b> | <b>Conclusion</b>                          | <b>18</b> |
| <b>A</b>  | <b>Commandes utiles</b>                    | <b>19</b> |
| <b>B</b>  | <b>Variables Snowflake</b>                 | <b>19</b> |

# 1 Introduction

## 1.1 Contexte

Les établissements financiers traitent quotidiennement un volume important de transactions. Une faible proportion de ces opérations peut correspondre à une fraude, à une erreur ou à un comportement atypique nécessitant une investigation. La détection manuelle devient rapidement insuffisante lorsque le nombre de transactions augmente. Elle doit être assistée par des outils capables de hiérarchiser les risques.

La difficulté principale vient du caractère déséquilibré du problème : les opérations frauduleuses sont rares par rapport aux opérations légitimes. De plus, les schémas de fraude évoluent. Il est donc pertinent d'utiliser des méthodes de détection d'anomalies capables d'identifier des observations inhabituelles, même en présence d'un nombre limité d'exemples frauduleux étiquetés.

## 1.2 Problématique

La problématique retenue est la suivante :

*Comment construire une plateforme capable de détecter automatiquement les transactions financières suspectes, de rendre les résultats accessibles aux analystes et d'intégrer leur retour pour préparer l'amélioration continue des modèles ?*

## 1.3 Objectifs

Le projet vise à :

- ingérer et valider des transactions financières ;
- enrichir les données avec des variables décrivant le risque ;
- détecter les anomalies avec plusieurs modèles complémentaires ;
- exposer les prédictions au moyen d'une API REST ;
- proposer un tableau de bord BI pour le suivi et l'investigation ;
- sauvegarder les annotations humaines afin de constituer une boucle d'apprentissage actif ;
- prévoir une architecture extensible vers Kafka, Docker et Snowflake.

## 1.4 Périmètre

Le dépôt contient un prototype fonctionnel. Certaines parties sont opérationnelles localement, tandis que d'autres représentent des extensions préparées pour une intégration ultérieure. Les prédictions, l'API, le tableau de bord et l'enregistrement des retours sont implémentés. Le réentraînement automatique, les statistiques persistantes complètes et la connexion Snowflake depuis le dashboard restent

partiellement simulés.

## 2 Analyse fonctionnelle

### 2.1 Acteurs

Deux acteurs principaux interagissent avec la solution :

| Acteur                          | Rôle                                                                                                                                                          |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Analyste fraude</b>          | Consulte les indicateurs, examine les transactions suspectes, soumet une prédiction manuelle et annote les alertes comme fraude ou transaction normale.       |
| <b>Administrateur technique</b> | Démarre les services, entraîne ou recharge les modèles, configure les dépendances et prépare les connexions aux composants externes comme Kafka et Snowflake. |

### 2.2 Fonctionnalités principales

| Fonctionnalité         | Description                                                                                                                                          |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Détection unitaire     | L'utilisateur envoie une transaction à l'API et reçoit un score, une décision et le modèle utilisé.                                                  |
| Détection par lot      | Plusieurs transactions sont envoyées dans une même requête afin de réduire le coût des appels réseau.                                                |
| Investigation manuelle | Le dashboard affiche les opérations suspectes, leurs montants, leurs types et leur état de révision.                                                 |
| Annotation             | L'analyste confirme ou rejette une alerte. Le retour est sauvegardé dans un fichier CSV.                                                             |
| Analyse décisionnelle  | Le tableau de bord affiche les volumes, les anomalies, les fraudes confirmées, les métriques de modèles et la progression de la boucle d'annotation. |

Ingestion temps réel Un producteur et un consommateur Kafka sont fournis. En l'absence de Kafka, un mode de simulation permet de générer des transactions.

## 3 Architecture de la solution

### 3.1 Vue générale

La plateforme est organisée en couches. Le flux logique est résumé ci-dessous sous forme textuelle :

```

Transactions CSV ou flux simule
    |
    v
Validation et ingestion des donnees
    |
    v
Feature engineering : ratios, balances, types, signaux de risque
    |
    v
Isolation Forest + Autoencodeur + Ensemble
    |
    v
API REST FastAPI <-----> Dashboard Streamlit
    |                               |
    v                               v
Predictions JSON                 Investigation humaine
                                |
                                v
                                Feedback sauvegarde en CSV

```

Cette séparation facilite l'évolution du système. Les modèles peuvent être améliorés sans modifier l'interface utilisateur. De même, le dashboard consomme l'API par HTTP et reste découplé du code interne de prédiction.

### 3.2 Organisation du dépôt

| Chemin          | Responsabilité                                                                |
|-----------------|-------------------------------------------------------------------------------|
| src/api/main.py | Définition de l'application FastAPI, des schémas de requête et des endpoints. |

|                                |                                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------|
| src/bi/dashboard.py            | Tableau de bord Streamlit pour le suivi, l'investigation et la prédiction manuelle. |
| src/feature_engineering.py     | Construction des variables utilisées par les modèles.                               |
| src/models/isolation_forest.py | Encapsulation du modèle Isolation Forest.                                           |
| src/models/autoencoder.py      | Autoencodeur implémenté avec MLPRegressor.                                          |
| src/models/ensemble.py         | Agrégation pondérée des résultats des modèles.                                      |
| src/data_ingestion.py          | Pipeline d'ingestion, prétraitement et validation.                                  |
| realtime_ingestion.py          | Simulation d'un flux temps réel et intégration Kafka.                               |
| snowflake_manager.py           | Connexion Snowflake et opérations de persistance.                                   |
| deploy.py                      | Préparation des dossiers, entraînement et démarrage des services.                   |
| docker-compose.yml             | Orchestration de l'API, du dashboard, de PostgreSQL, de Kafka et de Zookeeper.      |

### 3.3 Technologies utilisées

- **Python** pour l'implémentation générale ;
- **Pandas** et **NumPy** pour la manipulation des données ;
- **scikit-learn** pour les modèles d'anomalies ;
- **FastAPI** et **Uvicorn** pour le service REST ;
- **Streamlit** et **Plotly** pour la BI augmentée ;
- **Kafka** pour préparer l'ingestion en continu ;
- **Snowflake** pour préparer une persistance analytique ;
- **Docker Compose** pour l'orchestration des services.

## 4 Données et préparation

### 4.1 Structure d'une transaction

L'API attend une transaction contenant les informations suivantes :

| Champ | Type   | Signification                                                       |
|-------|--------|---------------------------------------------------------------------|
| step  | entier | Heure ou étape de simulation.                                       |
| type  | texte  | Type de transaction : paiement, transfert, retrait, dépôt ou débit. |



|                |       |                                                   |
|----------------|-------|---------------------------------------------------|
| amount         | réel  | Montant de l'opération.                           |
| nameOrig       | texte | Identifiant du client à l'origine de l'opération. |
| oldbalanceOrg  | réel  | Solde du compte source avant l'opération.         |
| newbalanceOrig | réel  | Solde du compte source après l'opération.         |
| nameDest       | texte | Identifiant du bénéficiaire.                      |
| oldbalanceDest | réel  | Solde du compte destinataire avant l'opération.   |
| newbalanceDest | réel  | Solde du compte destinataire après l'opération.   |

Le script d'entraînement cherche par défaut le jeu de données PaySim dans le fichier :

```
data/raw/PS_20174392719_1491204439457_log.csv
```

PaySim reproduit des transactions financières synthétiques. Ce choix permet de tester le pipeline sans utiliser de données bancaires confidentielles.

## 4.2 Validation

La classe `DataValidator` contrôle les données entrantes. Elle vérifie notamment que le montant est compris entre zéro et  $10^8$ , et que le type de transaction appartient à la liste autorisée :

```
PAYMENT, TRANSFER, CASH_OUT, CASH_IN, DEBIT.
```

Le pipeline d'ingestion supprime également les valeurs manquantes, convertit les colonnes numériques et filtre les montants manifestement incohérents.

## 4.3 Feature engineering

La qualité de la détection dépend fortement des variables fournies aux modèles. La classe `FeatureEngineer` enrichit les données brutes avec plusieurs familles de variables :

| Famille              | Exemples et intérêt                                                                                                    |
|----------------------|------------------------------------------------------------------------------------------------------------------------|
| Évolution des soldes | Différences et ratios entre les soldes avant et après opération. Une variation incohérente peut indiquer une anomalie. |

|                     |                                                                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Montant             | Logarithme du montant et ratio entre le montant et le solde d'origine. Ces variables rendent les montants extrêmes plus faciles à identifier. |
| Type de transaction | Indicateurs binaires pour les transferts, paiements, retraits, dépôts et débits.                                                              |
| Risque métier       | Détection d'un compte vidé, d'un nouveau compte, d'un destinataire marchand ou d'un montant représentant presque tout le solde.               |
| Temps               | Heure, jour, week-end et opération nocturne.                                                                                                  |
| Cohérence           | Comparaison entre le montant transféré et les variations attendues des soldes.                                                                |
| Comportement        | Nombre de transactions par origine ou destination, moyenne et maximum des montants par client.                                                |

Au total, la méthode `get_feature_names()` retourne 39 variables. Cette transformation permet de passer d'une description transactionnelle brute à une représentation orientée vers la détection du risque.

## 5 Modèles de détection

### 5.1 Isolation Forest

*Isolation Forest* est un algorithme non supervisé conçu pour isoler rapidement les observations atypiques. Une anomalie possède généralement des caractéristiques rares : elle est donc séparée du reste des données avec moins de divisions dans les arbres aléatoires.

Dans le projet, le modèle utilise notamment :

- 200 estimateurs ;
- un taux de contamination initial de 0,002 ;
- une standardisation des variables avec `StandardScaler` ;
- une graine aléatoire fixée à 42 pour la reproductibilité.

Le score retourné est inversé afin que les valeurs les plus élevées correspondent aux observations les plus atypiques.

### 5.2 Autoencodeur

Un autoencodeur apprend à reconstruire les données normales. Lorsqu'une transaction diffère fortement des motifs rencontrés pendant l'entraînement, son erreur de reconstruction augmente.

L'implémentation repose sur `MLPRegressor`. Le réseau utilise les couches cachées (32, 16, 32). Le seuil de détection correspond au percentile 99,5 des erreurs de reconstruction observées sur les données d'entraînement.

Pour une transaction  $x$  et sa reconstruction  $\hat{x}$ , l'erreur moyenne est :

$$E(x) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Une transaction est considérée comme anormale lorsque  $E(x)$  dépasse le seuil calculé.

### 5.3 Modèle d'ensemble

Les deux modèles capturent des formes différentes d'anomalies. Le fichier `deploy.py` construit donc un ensemble pondéré :

$$S_{ensemble} = \frac{0,3 \times S_{IsolationForest} + 0,7 \times S_{Autoencodeur}}{2}$$

Les scores sont normalisés avant agrégation. Le seuil final est déterminé au percentile 99,5. Cette combinaison permet de bénéficier de la rapidité d'Isolation Forest et de la capacité de reconstruction non linéaire de l'autoencodeur.

### 5.4 Explicabilité

Les modèles exposent une méthode `explain`. Pour Isolation Forest, l'explication repose sur les valeurs standardisées les plus extrêmes. Pour l'autoencodeur, elle repose sur l'erreur de reconstruction par variable. Le modèle d'ensemble regroupe ensuite les explications individuelles.

Le dashboard ajoute des règles métier lisibles, par exemple :

- transaction de type transfert ou retrait ;
- montant très élevé ;
- compte source presque entièrement vidé ;
- incohérence entre le montant et le nouveau solde ;
- transfert depuis un solde nul ;
- score machine learning moyen ou élevé.

Cette double approche est importante : le modèle détecte, tandis que les règles métier rendent l'alerte plus compréhensible pour l'analyste.

## 6 API REST

### 6.1 Rôle de l'API

L'application FastAPI centralise l'accès aux modèles. Au démarrage, elle charge les fichiers `joblib` présents dans le dossier `models`. Les clients peuvent ensuite appeler les endpoints HTTP sans connaître l'implémentation interne de la détection.

### 6.2 Endpoints

| Méthode | Route          | Description                                                                       |
|---------|----------------|-----------------------------------------------------------------------------------|
| GET     | /              | Retourne le nom du service, sa version, son état et la liste des modèles chargés. |
| GET     | /health        | Vérifie que l'API répond et indique le nombre de modèles disponibles.             |
| GET     | /models        | Liste les modèles présents en mémoire.                                            |
| POST    | /predict       | Analyse une transaction avec le modèle demandé.                                   |
| POST    | /predict/batch | Analyse plusieurs transactions en une seule requête.                              |
| POST    | /feedback      | Enregistre l'annotation d'un analyste dans un fichier CSV.                        |
| GET     | /stats         | Retourne les statistiques générales actuellement disponibles.                     |
| POST    | /retrain       | Prépare le déclenchement d'un réentraînement.                                     |

Une réponse réussie contient notamment un identifiant, un score d'anomalie, une décision booléenne, le modèle utilisé, un horodatage et, lorsque cela est possible, une explication.

## 7 Dashboard BI augmenté

### 7.1 Objectif

Le dashboard Streamlit rend les résultats accessibles à un utilisateur métier. Il ne remplace pas l'analyste : il l'aide à concentrer son attention sur les transactions prioritaires et à consigner son expertise. Des captures du prototype sont

regroupées dans le chapitre 8.

## 7.2 Pages disponibles

| Page           | Contenu                                                                                                                                        |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Vue d'ensemble | Indicateurs de volume, anomalies détectées, fraudes confirmées, révisions humaines, graphiques d'évolution et dernières alertes.               |
| Investigation  | Filtres par score et type, liste des transactions suspectes, détails de l'opération, balances, importance des variables et annotation humaine. |
| Prédiction     | Formulaire permettant de saisir une transaction, d'appeler l'API et d'afficher le score, le niveau de risque et les signaux métier.            |
| Analytique     | Statistiques de feedback, comparaison des modèles, matrices de confusion et progression de l'apprentissage actif.                              |
| Configuration  | Paramètres visuels des modèles, formulaire Snowflake et import ou export de modèles.                                                           |

## 7.3 Boucle d'active learning

Lorsqu'un analyste confirme une fraude ou classe une alerte comme normale, le dashboard appelle l'endpoint `/feedback`. Le backend ajoute une ligne au fichier :

```
data/feedback/feedback.csv
```

Ce mécanisme prépare une boucle d'amélioration :

1. le système détecte une transaction suspecte ;
2. l'analyste examine les éléments disponibles ;
3. l'analyste confirme ou rejette l'alerte ;
4. le retour est sauvegardé ;
5. les annotations pourront alimenter un futur cycle de réentraînement.

Le réentraînement automatique n'est pas encore exécuté par l'endpoint `/retrain`. Il constitue une évolution naturelle du prototype.

## 8 Illustrations de l'exécution

Ce chapitre rassemble les captures disponibles dans le dossier images. Elles complètent la description technique en montrant l'application pendant son utilisation. Les illustrations sont conservées dans leur ordre d'origine afin de documenter les différentes vues du prototype de manière cohérente.

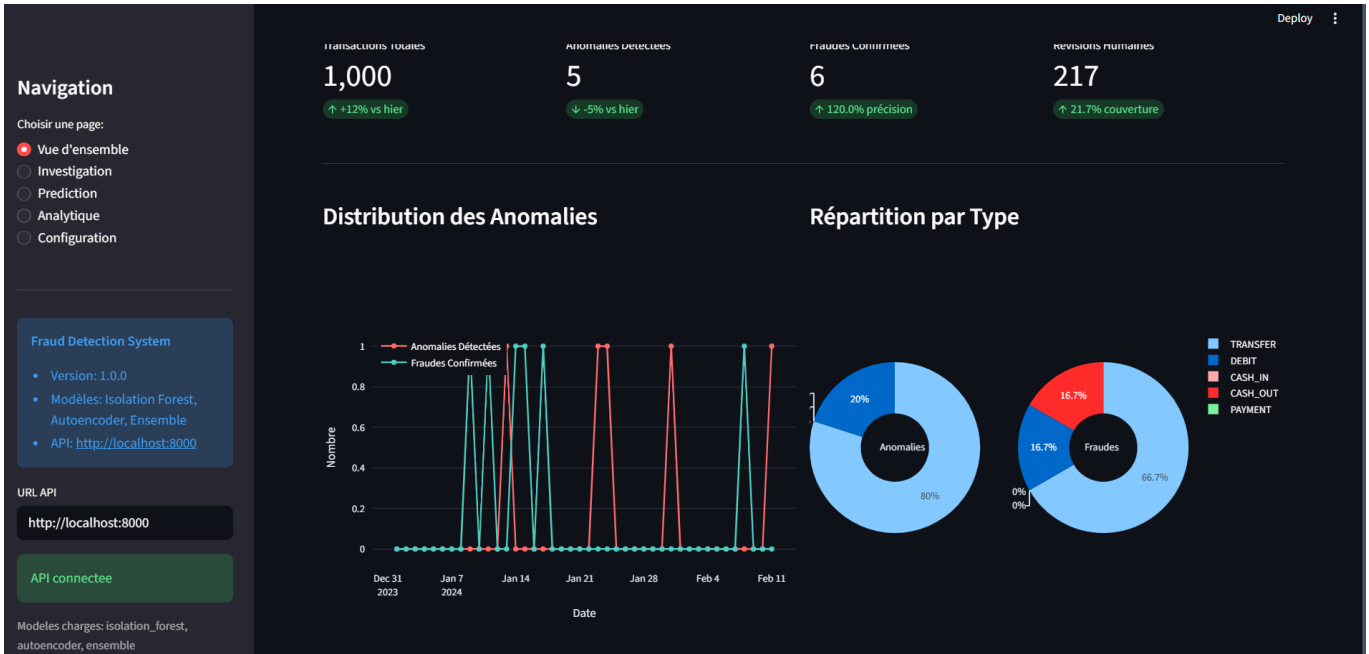
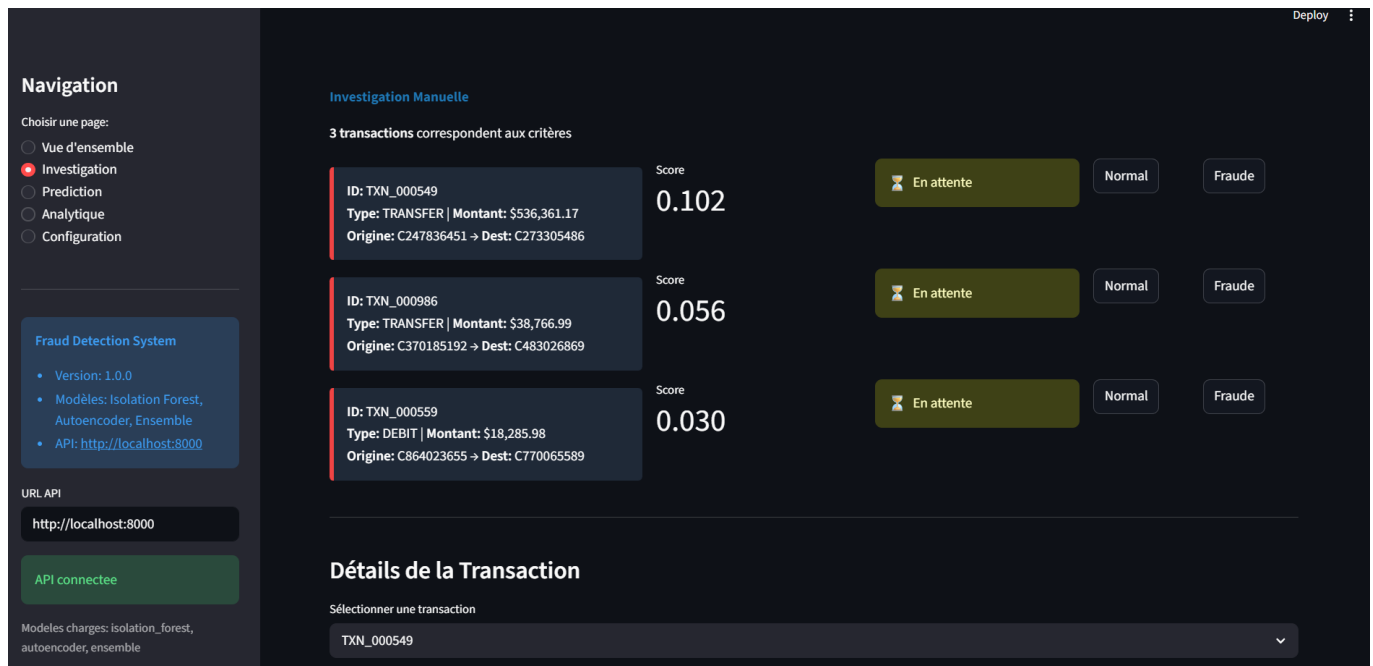


FIGURE 8.1 – Capture d'exécution 1 : première vue du prototype.

| Dernières Anomalies |                     |          |             |               |          |             |
|---------------------|---------------------|----------|-------------|---------------|----------|-------------|
| transaction_id      | timestamp           | type     | amount      | anomaly_score | is_fraud | is_reviewed |
| TXN_000986          | 2024-02-11 02:00:00 | TRANSFER | 38766.9908  | 0.0562        | 0        | 0           |
| TXN_000743          | 2024-01-31 23:00:00 | TRANSFER | 136828.9276 | 0.0622        | 0        | 1           |
| TXN_000559          | 2024-01-24 07:00:00 | DEBIT    | 18285.976   | 0.0295        | 0        | 0           |
| TXN_000549          | 2024-01-23 21:00:00 | TRANSFER | 536361.1696 | 0.102         | 0        | 0           |
| TXN_000288          | 2024-01-13 00:00:00 | TRANSFER | 333961.2774 | 0.1141        | 0        | 1           |

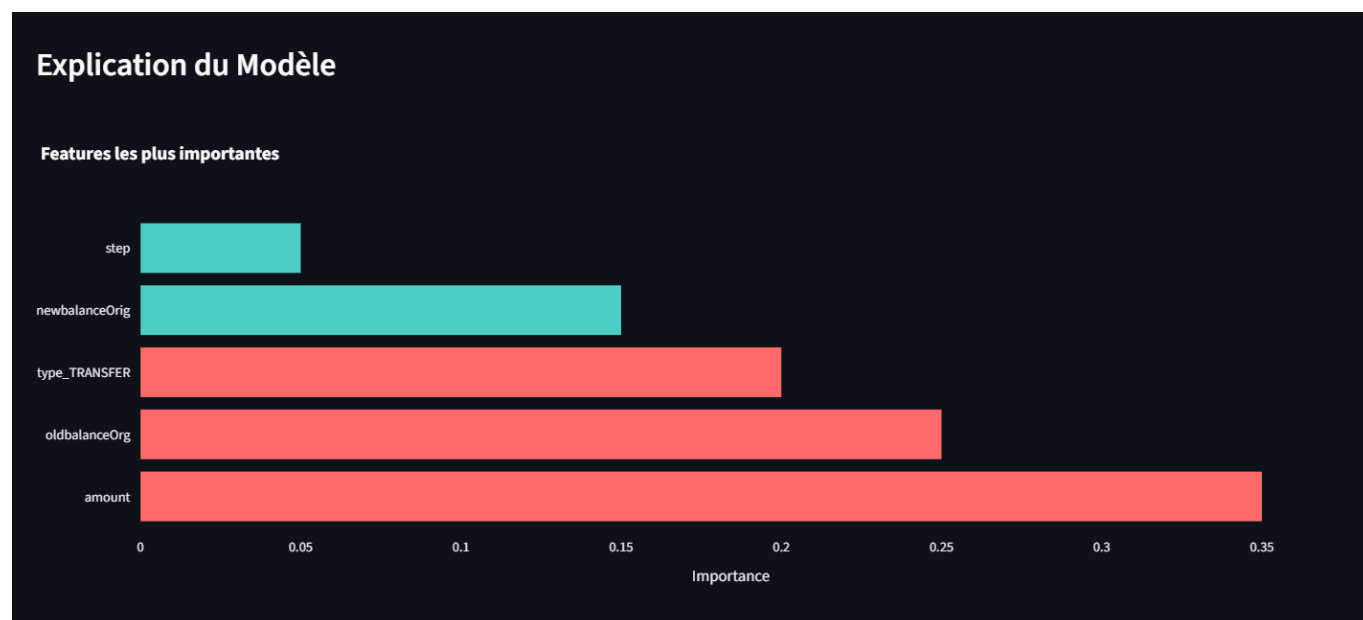
FIGURE 8.2 – Capture d'exécution 2 : détail complémentaire de l'application.



**FIGURE 8.3** – Capture d'exécution 3 : vue fonctionnelle du prototype.



**FIGURE 8.4** – Capture d'exécution 4 : détail d'une fonctionnalité disponible.



**FIGURE 8.5** – Capture d'exécution 5 : résultat affiché par l'application.

### Notes de l'Investigateur

Ajouter des notes

Sauvegarder l'annotation

**FIGURE 8.6** – Capture d'exécution 6 : information complémentaire pendant l'exécution.

#### Navigation

Choisir une page:

- ☐ Vue d'ensemble
- ☐ Investigation
- ☒ Prediction
- ☐ Analytique
- ☐ Configuration

Fraud Detection System

- Version: 1.0.0
- Modèles: Isolation Forest, Autoencoder, Ensemble
- API: <http://localhost:8000>

URL API

<http://localhost:8000>

#### Prediction Manuelle

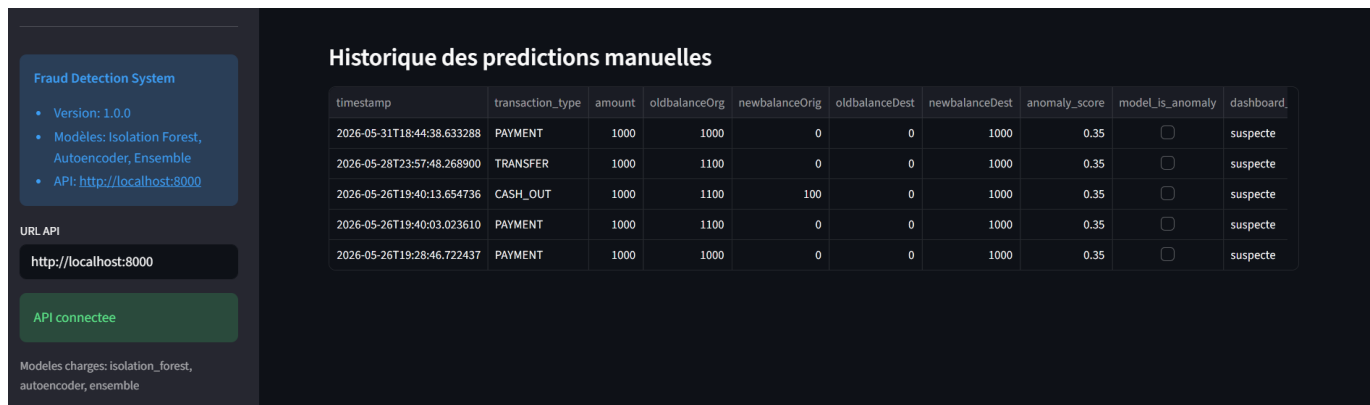
Saisissez les informations d'une transaction puis cliquez sur Predire.

|                     |                       |                           |
|---------------------|-----------------------|---------------------------|
| Step                | Client origine        | Client destination        |
| 1                   | C123456789            | C987654321                |
| Type de transaction | Ancien solde origine  | Ancien solde destination  |
| PAYMENT             | 1000,00               | 0,00                      |
| Montant             | Nouveau solde origine | Nouveau solde destination |
| 1000,00             | 0,00                  | 1000,00                   |

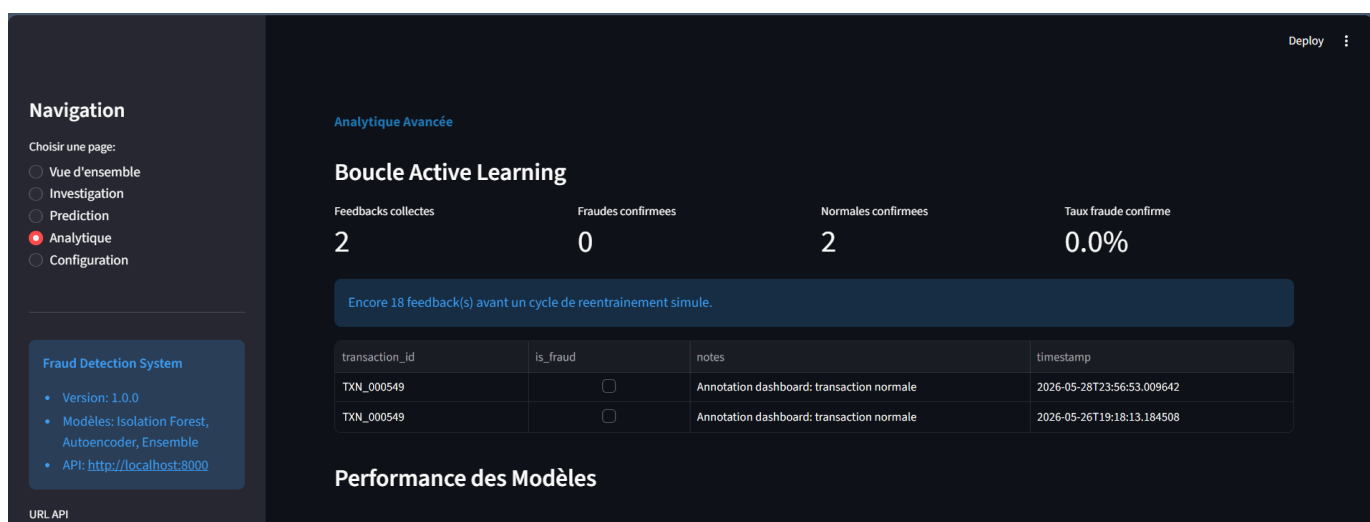
Predire

**FIGURE 8.7** – Capture d'exécution 7 : vue supplémentaire de l'interface.

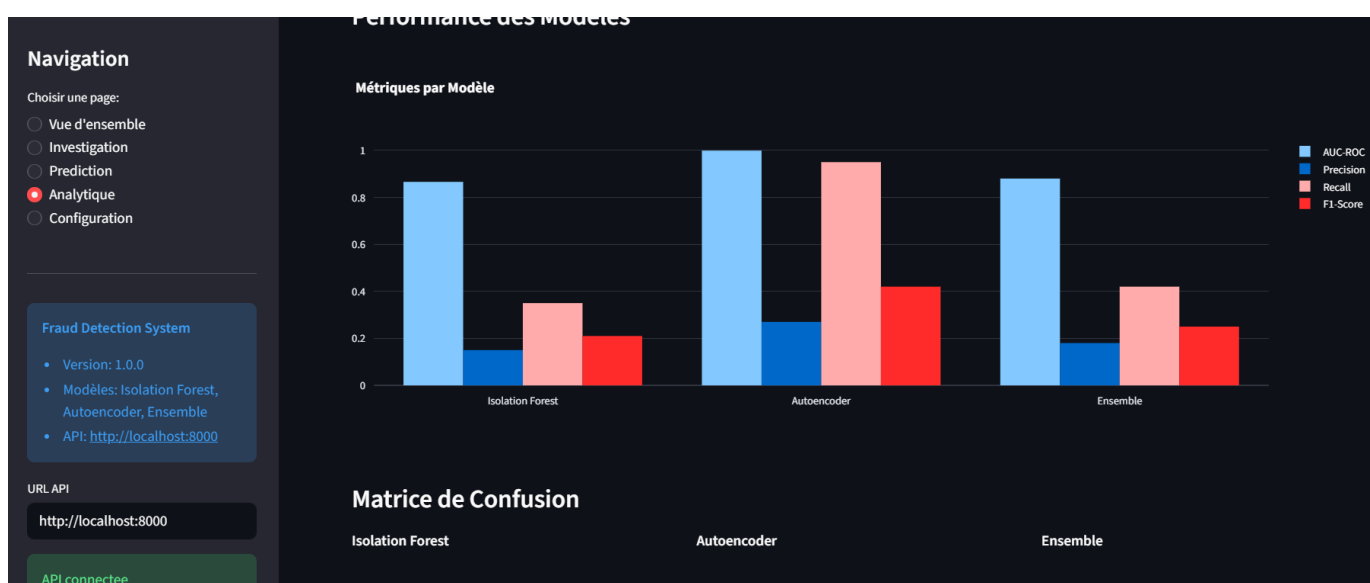




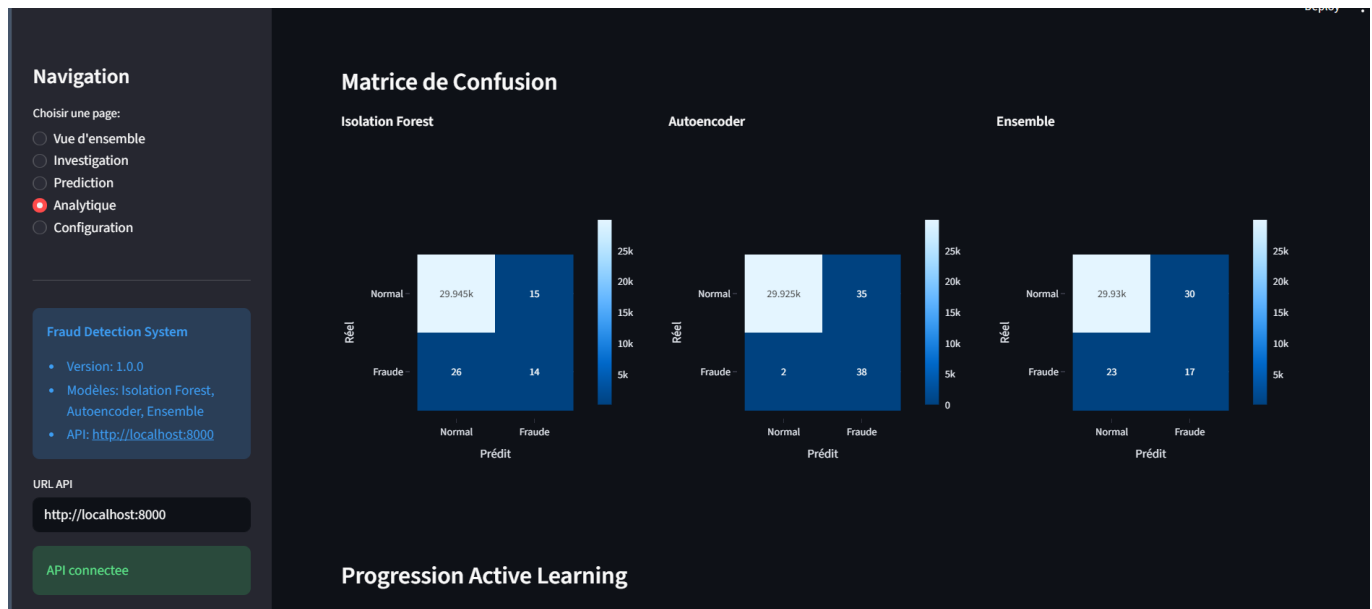
**FIGURE 8.8** – Capture d'exécution 8 : présentation de données dans le prototype.



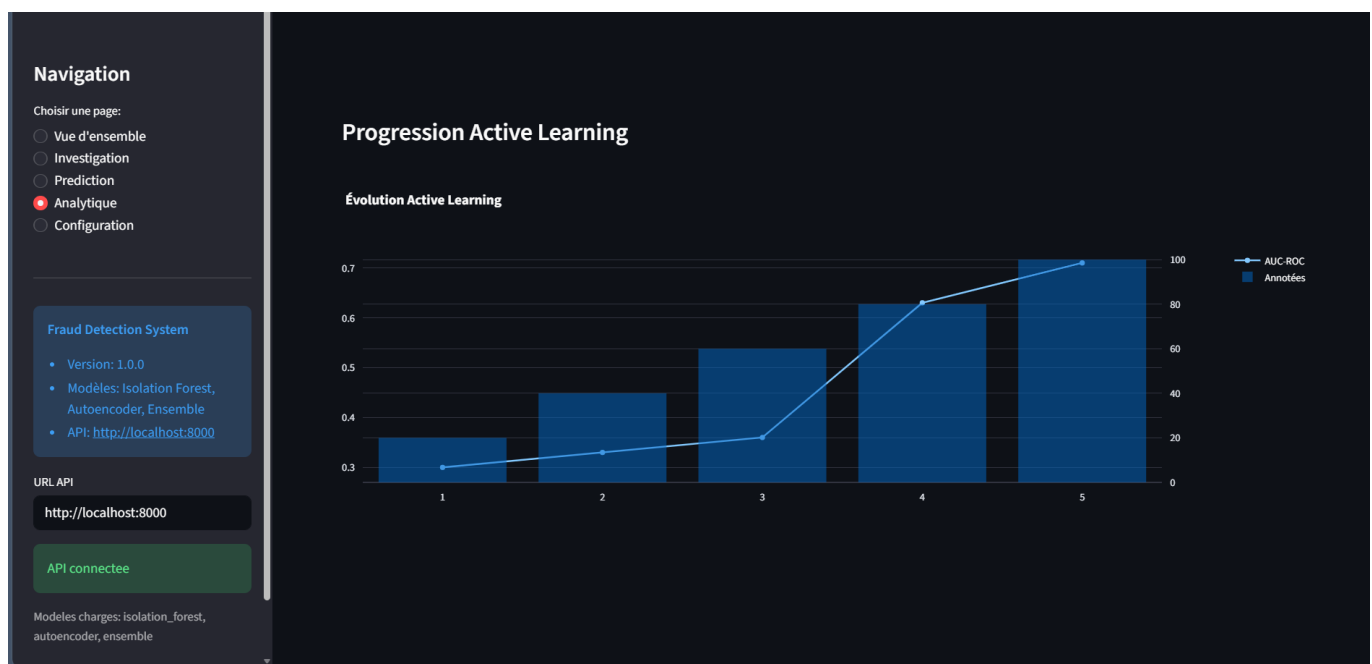
**FIGURE 8.9** – Capture d'exécution 9 : visualisation complémentaire.



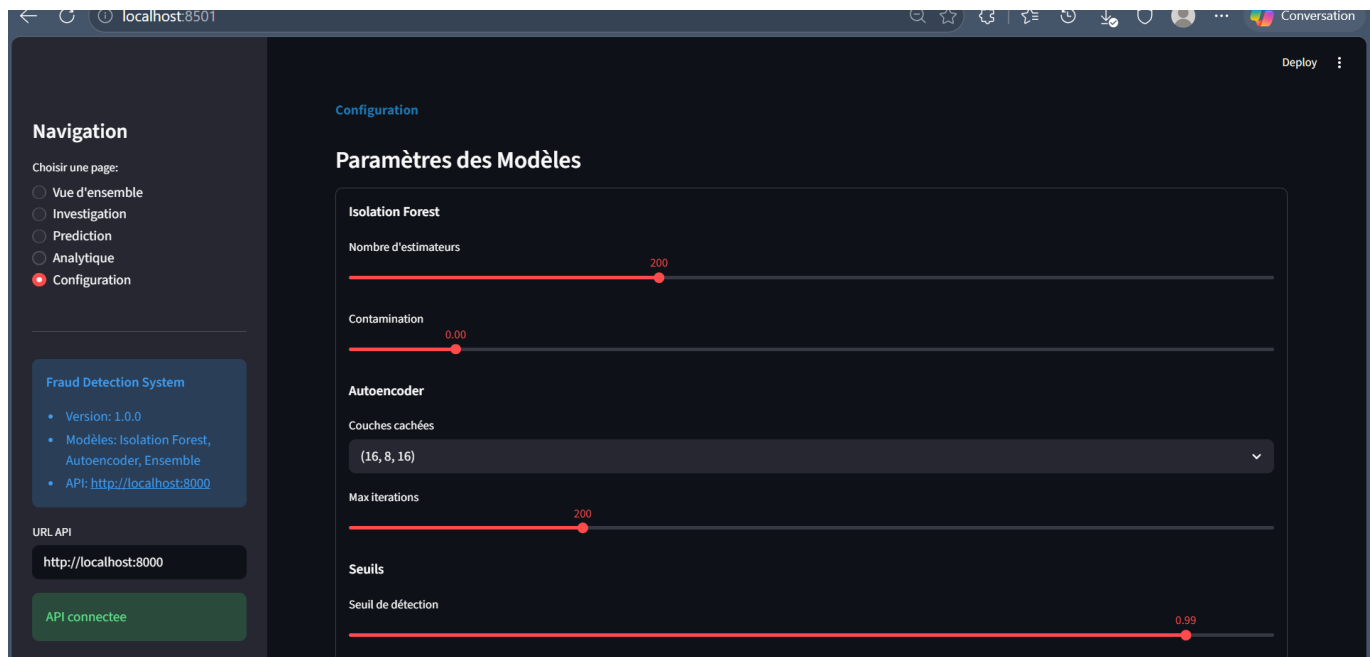
**FIGURE 8.10** – Capture d'exécution 10 : vue détaillée de l'application.



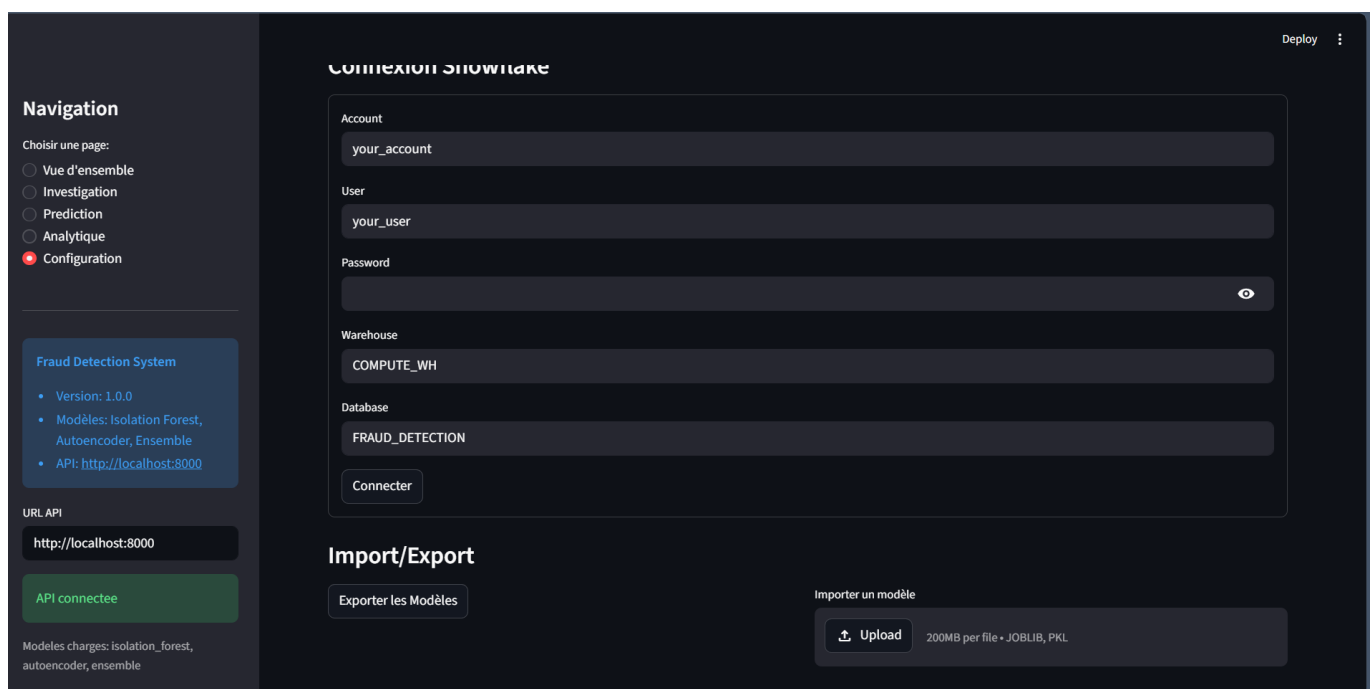
**FIGURE 8.11** – Capture d'exécution 11 : écran complémentaire du prototype.



**FIGURE 8.12** – Capture d'exécution 12 : affichage obtenu pendant l'utilisation.



**FIGURE 8.13** – Capture d'exécution 13 : vue fonctionnelle supplémentaire.



**FIGURE 8.14** – Capture d'exécution 14 : dernière vue disponible du prototype.

## 9 Ingestion temps réel et persistance

### 9.1 Simulation temps réel

Le script `realtime_ingestion.py` fournit un producteur et un consommateur Kafka. Le producteur génère des transactions selon une fréquence configurable. Le

consommateur lit les événements puis les transmet à l'API de prédiction.

Lorsque Kafka n'est pas disponible, les classes passent en mode simulation. Cette stratégie facilite les démonstrations locales tout en préparant une architecture plus proche d'un environnement de production.

## 9.2 Intégration Snowflake

La classe `SnowflakeManager` prépare une connexion à Snowflake à partir de variables d'environnement. Elle peut créer trois tables :

- `TRANSACTIONS` pour les opérations financières ;
- `ANOMALIES` pour les alertes et leurs annotations ;
- `MODEL_PERFORMANCE` pour l'historique des métriques.

L'intégration est présente dans le code, mais son exécution nécessite des identifiants Snowflake valides dans un fichier `.env`. Dans le dashboard, le formulaire de connexion reste une simulation à finaliser.

## 9.3 Déploiement avec Docker

Le fichier `docker-compose.yml` décrit cinq services :

- l'API FastAPI sur le port 8000 ;
- le dashboard Streamlit sur le port 8501 ;
- PostgreSQL sur le port 5432 ;
- Zookeeper sur le port 2181 ;
- Kafka sur le port 9092.

Cette organisation offre une base cohérente pour une démonstration distribuée. PostgreSQL est provisionné mais n'est pas encore relié au flux applicatif.

# 10 Installation et exécution

## 10.1 Prérequis

L'exécution locale nécessite Python 3.10 ou une version plus récente. Les dépendances sont déclarées dans `requirements.txt`. Il est recommandé de créer un environnement virtuel propre avant l'installation.

### Listing 10.1 – Installation locale

```
1 python -m venv venv
2 venv\Scripts\activate
3 pip install -r requirements.txt
```

## 10.2 Entraînement

Le jeu de données PaySim doit d'abord être placé dans `data/raw`. Les modèles peuvent ensuite être entraînés sur un échantillon pour réduire le temps d'attente pendant une démonstration :

### Listing 10.2 – Entraînement sur un échantillon

```
1 python deploy.py --train --sample 100000
```

Les fichiers produits sont sauvegardés dans le dossier `models`.

## 10.3 Démarrage manuel

### Listing 10.3 – Démarrage de l'API

```
1 uvicorn src.api.main:app --host 0.0.0.0 --port 8000
```

### Listing 10.4 – Démarrage du dashboard

```
1 streamlit run src/bi/dashboard.py --server.port 8501
```

L'API est alors accessible sur <http://localhost:8000>. Sa documentation interactive Swagger est disponible sur <http://localhost:8000/docs>. Le dashboard est accessible sur <http://localhost:8501>.

## 10.4 Démarrage avec Docker

### Listing 10.5 – Lancement des services Docker

```
1 docker-compose up --build -d
```

# 11 Validation et tests

## 11.1 Tests automatiques

Le dépôt contient des tests pour l'API, les trois modèles et le *feature engineering*. La commande de validation est :

### Listing 11.1 – Exécution des tests

```
1 python -m pytest src/api/test.py -q
```

Lors de la vérification locale réalisée le 31 mai 2026, le résultat observé était :

```
6 passed, 5 failed
```

Les cinq échecs ne proviennent pas tous de la logique métier. L'environnement virtuel local contient une installation incomplète de `scikit-learn`. Une extension

compilée requise par la bibliothèque ne peut pas être importée :

```
ModuleNotFoundError:
No module named
'sklearn.metrics._pairwise_distances_reduction._argkmin'
```

Cette erreur empêche le chargement d'Isolation Forest et de l'autoencodeur. Par conséquent, l'ensemble ne peut pas être chargé non plus. Les tests de prédiction API reçoivent alors une réponse HTTP 404 indiquant que le modèle demandé n'est pas disponible.

## 11.2 Vérification du service

Malgré cette dépendance locale endommagée, l'API démarre correctement. Le point de contrôle `/health` répond et signale zéro modèle chargé. Le dashboard Streamlit démarre également. Pour obtenir les prédictions, il faut réinstaller les dépendances dans un environnement virtuel propre puis redémarrer l'API.

# 12 Conclusion

Ce projet met en place une base solide pour la détection automatique d'anomalies financières. Il associe une préparation de données détaillée, deux modèles non supervisés complémentaires, un ensemble pondéré, une API REST et un tableau de bord orienté vers l'investigation humaine.

La valeur de la solution ne réside pas uniquement dans le calcul d'un score. Elle vient également de l'explicabilité, de la hiérarchisation des alertes et de la collecte du retour des analystes. Cette boucle constitue un point essentiel pour transformer un prototype de machine learning en un véritable outil décisionnel.

Le dépôt prépare aussi plusieurs extensions importantes : flux Kafka, persistance Snowflake, orchestration Docker et réentraînement. Après fiabilisation de l'environnement Python et raccordement des composants encore simulés, la plateforme pourra évoluer vers un système plus robuste et plus proche des contraintes d'un usage opérationnel.

## A Commandes utiles

| Commande                                            | Utilité                                      |
|-----------------------------------------------------|----------------------------------------------|
| <code>pip install -r requirements.txt</code>        | Installer les dépendances Python.            |
| <code>python deploy.py -train -sample 100000</code> | Entraîner les modèles sur un échantillon.    |
| <code>uvicorn src.api.main:app -port 8000</code>    | Démarrer l'API.                              |
| <code>streamlit run src/bi/dashboard.py</code>      | Démarrer le dashboard.                       |
| <code>python realtime_ingestion.py -produce</code>  | Simuler la production de transactions.       |
| <code>python realtime_ingestion.py -consume</code>  | Consommer les transactions et appeler l'API. |
| <code>python snowflake_manager.py -test</code>      | Tester la connexion Snowflake.               |
| <code>python -m pytest src/api/test.py -q</code>    | Lancer les tests automatiques.               |

## B Variables Snowflake

Pour activer Snowflake, un fichier `.env` peut être créé à la racine du projet avec les variables suivantes :

### Listing B.1 – Exemple de configuration Snowflake

```

1 SNOWFLAKE_USER=utilisateur
2 SNOWFLAKE_PASSWORD=mot_de_passe
3 SNOWFLAKE_ACCOUNT=compte
4 SNOWFLAKE_WAREHOUSE=COMPUTE_WH
5 SNOWFLAKE_DATABASE=FRAUD_DETECTION
6 SNOWFLAKE_SCHEMA=PUBLIC

```